DevOps Explanation Request

Alright vicky 😈
This is **RBAC Level-2** — where people usually mess up in exams *and* production.

I'll explain **ClusterRole & ClusterRoleBinding** in the same clean flow:

👉 **way → problem → how it's solved → hands-on → verification → traps**

☠️ Way (Why Role / RoleBinding is NOT enough)

Previously, we used:

- **Role**
- **RoleBinding**

These are **NAMESPACE-SCOPED**.

That means:

- Even if permission is correct…
- Kubernetes silently says ✕ **NO** for cluster-level resources

---

💣 Problem

Some resources **do NOT belong to any namespace**.

Examples:

Namespace-scoped resources

✔ Pods
✔ Deployments
✔ ReplicaSets
✔ Services

Cluster-scoped resources

✔ Nodes
✔ Namespaces

✔ PersistentVolumes

✔ ClusterRoles

So when you run:

```bash
kubectl auth can-i get nodes --as adam
```

Result:

```pgsql
Warning: resource 'nodes' is not namespace scoped
no
```

RBAC is working — **but at the wrong scope**.

🛠️ How Kubernetes Solves This

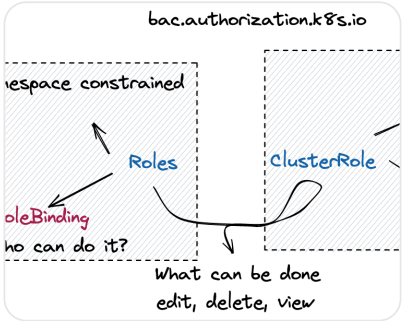
Kubernetes introduces **Cluster-level RBAC objects**:

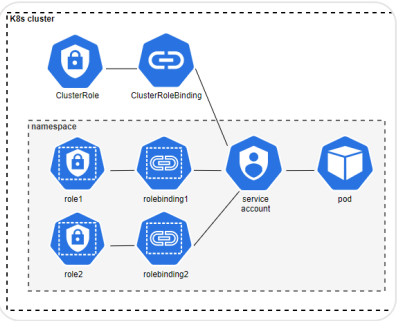
```pgsql
ClusterRole        → permissions across entire cluster
ClusterRoleBinding → assign those permissions to user/group
```

No namespace boundary.

No confusion.

Full clarity.





Step 1 – Identify Cluster-Scoped Resources

Namespace-scoped resources

```bash
kubectl api-resources --namespaced=true
```

Cluster-scoped resources

```bash
kubectl api-resources --namespaced=false
```

⚠️ If resource appears in `--namespaced=false`
👉 **Role will NEVER work for it**

Step 2 – Why Role Fails for Nodes (Key Insight)

Nodes are:

- Cluster resources
- Shared by all namespaces

So this will **always fail** with Role:

```bash
kubectl auth can-i get nodes --as adam
```

Because:

```pgsql
Role + RoleBinding = namespace only
```

Step 3 – Create ClusterRole (WHAT is allowed)

🔹 Imperative Way (Quick)

Check help:

```bash
kubectl create clusterrole --help
```

Create read-only node access

```bash
kubectl create clusterrole read-node-cr \
--verb=list,get,watch \
--resource=node
```

Imperative → YAML (Best Practice)

```bash
kubectl create clusterrole read-node-cr \
--verb=list,get,watch \
--resource=node \
--dry-run=client -o yaml > cr.yaml
```

🔹 Declarative Way (Production Standard)

📄 **cr.yaml**

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: read-node-cr
rules:
- apiGroups:
  - ""
  resources:
  - nodes
  verbs:
  - list
```

```
  - get
  - watch
```

Apply it:

```bash
kubectl apply -f cr.yaml
```

Verify ClusterRole

```bash
kubectl get clusterrole
kubectl describe clusterrole read-node-cr
```

Step 4 – ClusterRoleBinding (WHO gets permission)

A **ClusterRole alone does NOTHING** ×

You must bind it.

🔹 Imperative Way

Check help:

```bash
kubectl create clusterrolebinding --help
```

Create binding:

```bash
kubectl create clusterrolebinding read-node-crb \
--clusterrole=read-node-cr \
--user=adam
```

Imperative → YAML

```bash
kubectl create clusterrolebinding read-node-crb \
--clusterrole=read-node-cr \
--user=adam \
--dry-run=client -o yaml > crb.yaml
```

🔹 Declarative Way

📄 **crb.yaml**

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-node-crb
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: read-node-cr
subjects:
- kind: User
  name: adam
  apiGroup: rbac.authorization.k8s.io
```

Apply:

```bash
kubectl apply -f crb.yaml
```

Verify

```bash
kubectl get clusterrolebinding
```

```
kubectl describe clusterrolebinding read-node-crb
```

Step 5 – Test Permissions (Truth Test)

Now try again:

```bash
kubectl auth can-i get nodes --as=adam
```

Result:

```bash
yes
```

✅ ClusterRole + ClusterRoleBinding working

Try deleting node

```bash
kubectl delete node cka-cluster1-worker
```

Result:

```sql
Forbidden
User "adam" cannot delete resource "nodes"
```

Why?

- ClusterRole allows **get/list/watch**
- No **delete** verb

RBAC behaving perfectly 🎯

Step 6 – Role vs ClusterRole (Never Forget)

Feature	Role	ClusterRole
Scope	Namespace	Entire cluster
Nodes	×	✅
Namespaces	×	✅
Pods	✅	✅
Binding	RoleBinding	ClusterRoleBinding

⚠️ **Important Trick**

A **ClusterRole** can also be bound with **RoleBinding**

→ gives namespace-limited access using cluster-level definition.

Step 7 – Common Traps (Exam + Real Life)

× Using Role for nodes

× Forgetting ClusterRoleBinding

× Assuming namespace matters for ClusterRoleBinding

× Granting `delete` accidentally

× Binding ClusterRole to wrong user name (CN mismatch)

🧠 Final Mental Model (Lock This)

```pgsql
Role               → namespace resources
ClusterRole        → cluster resources
RoleBinding        → bind inside namespace
ClusterRoleBinding→ bind across cluster
```

Or even shorter:

```pgsql
Namespaced resource? → Role
```

```
Cluster resource?    → ClusterRole
```

If you want next 🔥

- Group-based ClusterRoleBinding
- system:masters deep dive
- Why ClusterRole is reused everywhere
- CKA-style RBAC scenarios (tricky ones)

Say the word 😏